READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Mounted at /content/drive
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))
```

```
all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")

print(f"pneumon : {len(all_pneumonia_files)}")
```

```
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

BALANCING

```
train_pneumonia_count = 3200
test_pneumonia_count = 1065
train_normal_count = 3200
test_normal_count = 1065

test_pneumonia_samples = random.sample(all_pneumonia_files, test_pneumonia_count)
test_normal_samples = random.sample(all_normal_files, test_normal_count)

pneumonia_train_pool = [f for f in all_pneumonia_files if f not in test_pneumonia_samples]
normal_train_pool = [f for f in all_normal_files if f not in test_normal_samples]

train_pneumonia_samples = random.choices(pneumonia_train_pool, k=train_pneumonia_count)
train_normal_samples = random.choices(normal_train_pool, k=train_normal_count)
```

```
print(f"train_pneumonia_samples : {len(train_pneumonia_samples)}")
print(f"test_pneumonia_samples : {len(test_pneumonia_samples)}")
print(f"train_normal_samples : {len(train_normal_samples)}")
print(f"test_normal_samples : {len(test_normal_samples)}")
```

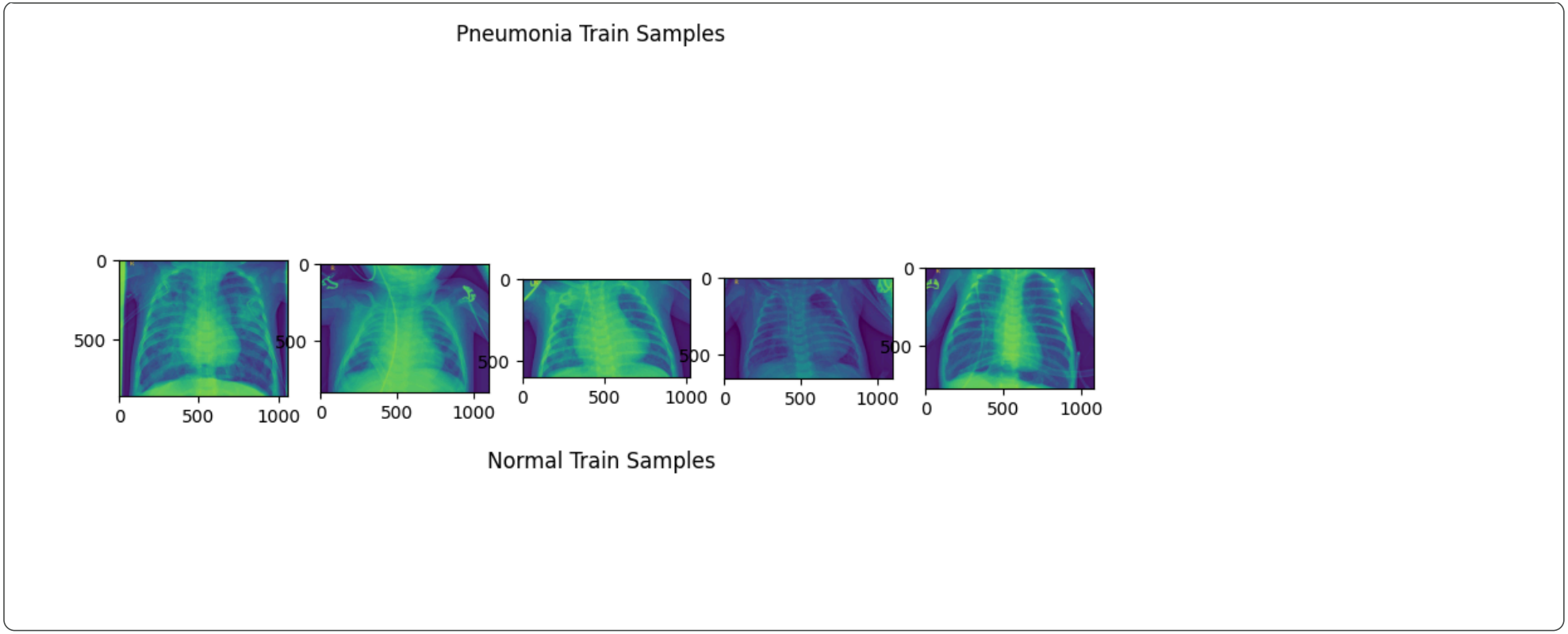
```
train_pneumonia_samples : 3200
test_pneumonia_samples : 1065
train_normal_samples : 3200
test_normal_samples : 1065
```

```
from PIL import Image
import matplotlib.pyplot as plt

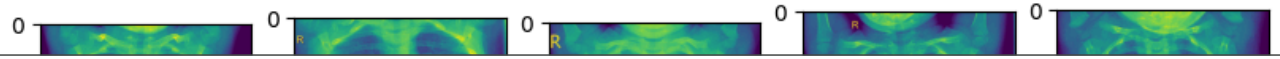
def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
```

```
            img = Image.open(sample_files[i])
            plt.imshow(img)
        plt.show()

    show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
    show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
    show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
    show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```



RESIZE



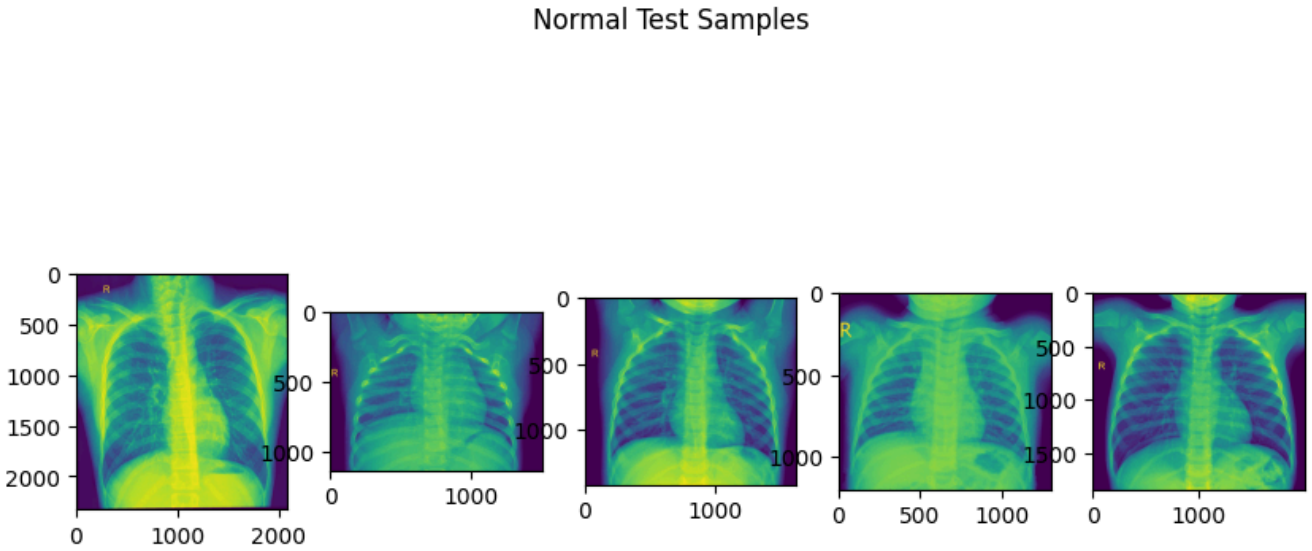
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

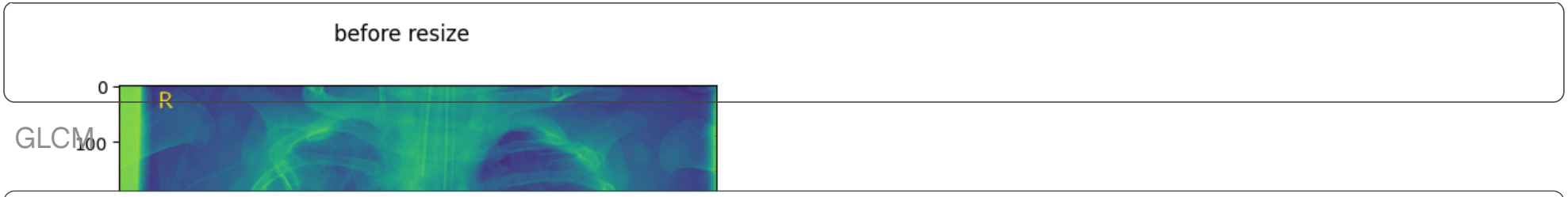
def load_images(file_paths, target_size=(224, 224)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
```

```
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)
```

```
show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```





GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```



```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
```

```
            glcm_features_list.append(features)
        return glcm_features_list

    train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
    test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
    train_normal_glcm = extract_glcm_features(train_normal_images)
    test_normal_glcm = extract_glcm_features(test_normal_images)

    print("GLCM features for the first pneumonia training image:")
    train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(1215.7637282097342),
 'contrast_std': np.float64(604.2551625381589),
 'contrast_var': np.float64(365124.30145401676),
 'dissimilarity_mean': np.float64(13.446299555270656),
 'dissimilarity_std': np.float64(4.462842562958955),
 'dissimilarity_var': np.float64(19.916963741758057),
 'homogeneity_mean': np.float64(0.16560993036757773),
 'homogeneity_std': np.float64(0.04336721745535526),
 'homogeneity_var': np.float64(0.0018807155498200701),
 'energy_mean': np.float64(0.014852845892011206),
 'energy_std': np.float64(0.001976914214063014),
 'energy_var': np.float64(3.908189809764385e-06),
 'correlation_mean': np.float64(0.8109860402664245),
 'correlation_std': np.float64(0.09408777855407914),
 'correlation_var': np.float64(0.008852510073241435),
 'ASM_mean': np.float64(0.00022451522090159858),
 'ASM_std': np.float64(6.113779159230574e-05),
 'ASM_var': np.float64(3.7378295607842105e-09),
 'entropy': np.float64(153.51413650128237)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'

#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'
```

```
#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)


# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    #Dense(64, activation='relu', kernel_regularizer=l2(0.0001)),
    #BatchNormalization(),
    #Dropout(0.3),
    #Dense(32, activation='relu', kernel_regularizer=l2(0.0001)),
    #BatchNormalization(),
    #Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=50, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    #epochs=50,
    epochs=100,
    batch_size=32,
    validation_split=0.3,
    #callbacks=None,
    callbacks=callbacks,
    class_weight=class_weights,
)
```



```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 128)	2,560
batch_normalization_3 (BatchNormalization)	(None, 128)	512
dropout_3 (Dropout)	(None, 128)	0
dense_5 (Dense)	(None, 1)	129

```
 Total params: 3,201 (12.50 KB)
 Trainable params: 2,945 (11.50 KB)
 Non-trainable params: 256 (1.00 KB)
Epoch 1/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 7s 18ms/step - accuracy: 0.8188 - auc_1: 0.8958 - loss: 0.4246 - val_accuracy: 0.5818 - val_auc_1: 0.0000e+00 - val_loss: 0.736
Epoch 2/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - accuracy: 0.8701 - auc_1: 0.9278 - loss: 0.3381 - val_accuracy: 0.8240 - val_auc_1: 0.0000e+00 - val_loss: 0.51
Epoch 3/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8732 - auc_1: 0.9330 - loss: 0.3265 - val_accuracy: 0.8750 - val_auc_1: 0.0000e+00 - val_loss: 0.388
Epoch 4/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8714 - auc_1: 0.9348 - loss: 0.3239 - val_accuracy: 0.9005 - val_auc_1: 0.0000e+00 - val_loss: 0.273
Epoch 5/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8725 - auc_1: 0.9376 - loss: 0.3163 - val_accuracy: 0.8833 - val_auc_1: 0.0000e+00 - val_loss: 0.324
Epoch 6/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8855 - auc_1: 0.9440 - loss: 0.2994 - val_accuracy: 0.9036 - val_auc_1: 0.0000e+00 - val_loss: 0.275
Epoch 7/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8853 - auc_1: 0.9459 - loss: 0.2938 - val_accuracy: 0.8948 - val_auc_1: 0.0000e+00 - val_loss: 0.296
Epoch 8/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8857 - auc_1: 0.9458 - loss: 0.2960 - val_accuracy: 0.8875 - val_auc_1: 0.0000e+00 - val_loss: 0.323
Epoch 9/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8871 - auc_1: 0.9479 - loss: 0.2890 - val_accuracy: 0.8896 - val_auc_1: 0.0000e+00 - val_loss: 0.305
Epoch 10/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8893 - auc_1: 0.9500 - loss: 0.2823 - val_accuracy: 0.8859 - val_auc_1: 0.0000e+00 - val_loss: 0.305
Epoch 11/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8908 - auc_1: 0.9478 - loss: 0.2884 - val_accuracy: 0.8943 - val_auc_1: 0.0000e+00 - val_loss: 0.297
Epoch 12/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8922 - auc_1: 0.9513 - loss: 0.2788 - val_accuracy: 0.9083 - val_auc_1: 0.0000e+00 - val_loss: 0.258
Epoch 13/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8960 - auc_1: 0.9523 - loss: 0.2777 - val_accuracy: 0.8948 - val_auc_1: 0.0000e+00 - val_loss: 0.293
Epoch 14/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8958 - auc_1: 0.9526 - loss: 0.2747 - val_accuracy: 0.8974 - val_auc_1: 0.0000e+00 - val_loss: 0.287
Epoch 15/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8955 - auc_1: 0.9529 - loss: 0.2742 - val_accuracy: 0.9005 - val_auc_1: 0.0000e+00 - val_loss: 0.284
Epoch 16/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8893 - auc_1: 0.9511 - loss: 0.2793 - val_accuracy: 0.8766 - val_auc_1: 0.0000e+00 - val_loss: 0.314
Epoch 17/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8935 - auc_1: 0.9543 - loss: 0.2699 - val_accuracy: 0.9094 - val_auc_1: 0.0000e+00 - val_loss: 0.250
```

```
Epoch 18/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8953 - auc_1: 0.9533 - loss: 0.2736 - val_accuracy: 0.9016 - val_auc_1: 0.0000e+00 - val_loss: 0.269
Epoch 19/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8962 - auc_1: 0.9570 - loss: 0.2640 - val_accuracy: 0.8604 - val_auc_1: 0.0000e+00 - val_loss: 0.347
Epoch 20/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9004 - auc_1: 0.9561 - loss: 0.2638 - val_accuracy: 0.9125 - val_auc_1: 0.0000e+00 - val_loss: 0.243
Epoch 21/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9029 - auc_1: 0.9589 - loss: 0.2569 - val_accuracy: 0.9089 - val_auc_1: 0.0000e+00 - val_loss: 0.262
Epoch 22/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8993 - auc_1: 0.9570 - loss: 0.2628 - val_accuracy: 0.8958 - val_auc_1: 0.0000e+00 - val_loss: 0.265
Epoch 23/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8946 - auc_1: 0.9567 - loss: 0.2634 - val_accuracy: 0.9036 - val_auc_1: 0.0000e+00 - val_loss: 0.282
Epoch 24/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8991 - auc_1: 0.9577 - loss: 0.2625 - val_accuracy: 0.9005 - val_auc_1: 0.0000e+00 - val_loss: 0.286
Epoch 25/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8949 - auc_1: 0.9568 - loss: 0.2653 - val_accuracy: 0.9031 - val_auc_1: 0.0000e+00 - val_loss: 0.251
Epoch 26/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9049 - auc_1: 0.9602 - loss: 0.2523 - val_accuracy: 0.9094 - val_auc_1: 0.0000e+00 - val_loss: 0.258
Epoch 27/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8982 - auc_1: 0.9582 - loss: 0.2601 - val_accuracy: 0.9089 - val_auc_1: 0.0000e+00 - val_loss: 0.260
Epoch 28/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9076 - auc_1: 0.9629 - loss: 0.2447 - val_accuracy: 0.9026 - val_auc_1: 0.0000e+00 - val_loss: 0.278
Epoch 29/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9038 - auc_1: 0.9596 - loss: 0.2539 - val_accuracy: 0.9130 - val_auc_1: 0.0000e+00 - val_loss: 0.262
Epoch 30/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9060 - auc_1: 0.9629 - loss: 0.2443 - val_accuracy: 0.8984 - val_auc_1: 0.0000e+00 - val_loss: 0.262
Epoch 31/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9094 - auc_1: 0.9608 - loss: 0.2507 - val_accuracy: 0.9135 - val_auc_1: 0.0000e+00 - val_loss: 0.245
Epoch 32/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9060 - auc_1: 0.9622 - loss: 0.2469 - val_accuracy: 0.9078 - val_auc_1: 0.0000e+00 - val_loss: 0.254
Epoch 33/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9049 - auc_1: 0.9617 - loss: 0.2492 - val_accuracy: 0.9271 - val_auc_1: 0.0000e+00 - val_loss: 0.243
Epoch 34/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9078 - auc_1: 0.9637 - loss: 0.2422 - val_accuracy: 0.9073 - val_auc_1: 0.0000e+00 - val_loss: 0.289
Epoch 35/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9002 - auc_1: 0.9603 - loss: 0.2527 - val_accuracy: 0.8990 - val_auc_1: 0.0000e+00 - val_loss: 0.272
Epoch 36/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9067 - auc_1: 0.9626 - loss: 0.2447 - val_accuracy: 0.9120 - val_auc_1: 0.0000e+00 - val_loss: 0.283
Epoch 37/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9025 - auc_1: 0.9614 - loss: 0.2492 - val_accuracy: 0.9297 - val_auc_1: 0.0000e+00 - val_loss: 0.243
Epoch 38/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9022 - auc_1: 0.9616 - loss: 0.2473 - val_accuracy: 0.9141 - val_auc_1: 0.0000e+00 - val_loss: 0.226
Epoch 39/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9087 - auc_1: 0.9654 - loss: 0.2366 - val_accuracy: 0.8729 - val_auc_1: 0.0000e+00 - val_loss: 0.293
Epoch 40/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9045 - auc_1: 0.9614 - loss: 0.2502 - val_accuracy: 0.9281 - val_auc_1: 0.0000e+00 - val_loss: 0.245
Epoch 41/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9074 - auc_1: 0.9633 - loss: 0.2420 - val_accuracy: 0.9135 - val_auc_1: 0.0000e+00 - val_loss: 0.242
Epoch 42/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9103 - auc_1: 0.9639 - loss: 0.2416 - val_accuracy: 0.8932 - val_auc_1: 0.0000e+00 - val_loss: 0.276
Epoch 43/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9116 - auc_1: 0.9653 - loss: 0.2362 - val_accuracy: 0.9156 - val_auc_1: 0.0000e+00 - val_loss: 0.248
Epoch 44/100
```

```
Epoch 44/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9060 - auc_1: 0.9637 - loss: 0.2434 - val_accuracy: 0.9161 - val_auc_1: 0.0000e+00 - val_loss: 0.245
Epoch 45/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9040 - auc_1: 0.9595 - loss: 0.2552 - val_accuracy: 0.8917 - val_auc_1: 0.0000e+00 - val_loss: 0.284
Epoch 46/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9062 - auc_1: 0.9627 - loss: 0.2458 - val_accuracy: 0.8885 - val_auc_1: 0.0000e+00 - val_loss: 0.291
Epoch 47/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9067 - auc_1: 0.9630 - loss: 0.2450 - val_accuracy: 0.8917 - val_auc_1: 0.0000e+00 - val_loss: 0.300
Epoch 48/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9049 - auc_1: 0.9621 - loss: 0.2470 - val_accuracy: 0.9000 - val_auc_1: 0.0000e+00 - val_loss: 0.275
Epoch 49/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9112 - auc_1: 0.9658 - loss: 0.2363 - val_accuracy: 0.8927 - val_auc_1: 0.0000e+00 - val_loss: 0.258
Epoch 50/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9058 - auc_1: 0.9660 - loss: 0.2333 - val_accuracy: 0.9193 - val_auc_1: 0.0000e+00 - val_loss: 0.248
Epoch 51/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9065 - auc_1: 0.9638 - loss: 0.2412 - val_accuracy: 0.9146 - val_auc_1: 0.0000e+00 - val_loss: 0.239
Epoch 52/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9074 - auc_1: 0.9620 - loss: 0.2462 - val_accuracy: 0.9161 - val_auc_1: 0.0000e+00 - val_loss: 0.246
Epoch 53/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9020 - auc_1: 0.9630 - loss: 0.2443 - val_accuracy: 0.9198 - val_auc_1: 0.0000e+00 - val_loss: 0.228
Epoch 54/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9040 - auc_1: 0.9636 - loss: 0.2432 - val_accuracy: 0.9240 - val_auc_1: 0.0000e+00 - val_loss: 0.216
Epoch 55/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9071 - auc_1: 0.9640 - loss: 0.2421 - val_accuracy: 0.9417 - val_auc_1: 0.0000e+00 - val_loss: 0.207
Epoch 56/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9125 - auc_1: 0.9652 - loss: 0.2366 - val_accuracy: 0.9052 - val_auc_1: 0.0000e+00 - val_loss: 0.260
Epoch 57/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9123 - auc_1: 0.9648 - loss: 0.2381 - val_accuracy: 0.9177 - val_auc_1: 0.0000e+00 - val_loss: 0.233
Epoch 58/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9156 - auc_1: 0.9658 - loss: 0.2340 - val_accuracy: 0.9281 - val_auc_1: 0.0000e+00 - val_loss: 0.217
Epoch 59/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9076 - auc_1: 0.9659 - loss: 0.2346 - val_accuracy: 0.9135 - val_auc_1: 0.0000e+00 - val_loss: 0.249
Epoch 60/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9103 - auc_1: 0.9655 - loss: 0.2346 - val_accuracy: 0.9104 - val_auc_1: 0.0000e+00 - val_loss: 0.246
Epoch 61/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9109 - auc_1: 0.9647 - loss: 0.2408 - val_accuracy: 0.9302 - val_auc_1: 0.0000e+00 - val_loss: 0.238
Epoch 62/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9125 - auc_1: 0.9646 - loss: 0.2385 - val_accuracy: 0.9339 - val_auc_1: 0.0000e+00 - val_loss: 0.207
Epoch 63/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9150 - auc_1: 0.9670 - loss: 0.2306 - val_accuracy: 0.9214 - val_auc_1: 0.0000e+00 - val_loss: 0.230
Epoch 64/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9136 - auc_1: 0.9697 - loss: 0.2218 - val_accuracy: 0.9141 - val_auc_1: 0.0000e+00 - val_loss: 0.236
Epoch 65/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9143 - auc_1: 0.9691 - loss: 0.2250 - val_accuracy: 0.9156 - val_auc_1: 0.0000e+00 - val_loss: 0.253
Epoch 66/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9127 - auc_1: 0.9699 - loss: 0.2208 - val_accuracy: 0.9026 - val_auc_1: 0.0000e+00 - val_loss: 0.254
Epoch 67/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9143 - auc_1: 0.9685 - loss: 0.2260 - val_accuracy: 0.9255 - val_auc_1: 0.0000e+00 - val_loss: 0.223
Epoch 68/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9125 - auc_1: 0.9682 - loss: 0.2264 - val_accuracy: 0.9391 - val_auc_1: 0.0000e+00 - val_loss: 0.215
Epoch 69/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9176 - auc_1: 0.9681 - loss: 0.2269 - val_accuracy: 0.9146 - val_auc_1: 0.0000e+00 - val_loss: 0.218
Epoch 70/100
```

```
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9092 - auc_1: 0.9677 - loss: 0.2295 - val_accuracy: 0.9083 - val_auc_1: 0.0000e+00 - val_loss: 0.2514
Epoch 71/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9136 - auc_1: 0.9691 - loss: 0.2234 - val_accuracy: 0.8964 - val_auc_1: 0.0000e+00 - val_loss: 0.2514
Epoch 72/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9094 - auc_1: 0.9660 - loss: 0.2351 - val_accuracy: 0.9099 - val_auc_1: 0.0000e+00 - val_loss: 0.2418
Epoch 73/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9109 - auc_1: 0.9672 - loss: 0.2325 - val_accuracy: 0.8740 - val_auc_1: 0.0000e+00 - val_loss: 0.2906
Epoch 74/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9094 - auc_1: 0.9666 - loss: 0.2341 - val_accuracy: 0.9359 - val_auc_1: 0.0000e+00 - val_loss: 0.2074
Epoch 75/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9109 - auc_1: 0.9671 - loss: 0.2317 - val_accuracy: 0.9198 - val_auc_1: 0.0000e+00 - val_loss: 0.2254
Epoch 76/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9112 - auc_1: 0.9670 - loss: 0.2320 - val_accuracy: 0.9104 - val_auc_1: 0.0000e+00 - val_loss: 0.2314
Epoch 77/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9107 - auc_1: 0.9675 - loss: 0.2313 - val_accuracy: 0.9057 - val_auc_1: 0.0000e+00 - val_loss: 0.2457
Epoch 78/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9112 - auc_1: 0.9672 - loss: 0.2313 - val_accuracy: 0.9130 - val_auc_1: 0.0000e+00 - val_loss: 0.242
Epoch 79/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9121 - auc_1: 0.9674 - loss: 0.2308 - val_accuracy: 0.9177 - val_auc_1: 0.0000e+00 - val_loss: 0.213
Epoch 80/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9125 - auc_1: 0.9674 - loss: 0.2323 - val_accuracy: 0.9073 - val_auc_1: 0.0000e+00 - val_loss: 0.252
Epoch 81/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9121 - auc_1: 0.9704 - loss: 0.2198 - val_accuracy: 0.9234 - val_auc_1: 0.0000e+00 - val_loss: 0.226
Epoch 82/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9158 - auc_1: 0.9696 - loss: 0.2221 - val_accuracy: 0.9203 - val_auc_1: 0.0000e+00 - val_loss: 0.2188
Epoch 83/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9170 - auc_1: 0.9748 - loss: 0.2031 - val_accuracy: 0.9224 - val_auc_1: 0.0000e+00 - val_loss: 0.2248
Epoch 84/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9152 - auc_1: 0.9714 - loss: 0.2160 - val_accuracy: 0.9172 - val_auc_1: 0.0000e+00 - val_loss: 0.237
Epoch 85/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9217 - auc_1: 0.9734 - loss: 0.2076 - val_accuracy: 0.9172 - val_auc_1: 0.0000e+00 - val_loss: 0.220
Epoch 86/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9208 - auc_1: 0.9722 - loss: 0.2126 - val_accuracy: 0.9177 - val_auc_1: 0.0000e+00 - val_loss: 0.2274
Epoch 87/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9179 - auc_1: 0.9715 - loss: 0.2162 - val_accuracy: 0.9240 - val_auc_1: 0.0000e+00 - val_loss: 0.207
Epoch 88/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9179 - auc_1: 0.9724 - loss: 0.2127 - val_accuracy: 0.9182 - val_auc_1: 0.0000e+00 - val_loss: 0.2179
Epoch 89/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9187 - auc_1: 0.9715 - loss: 0.2166 - val_accuracy: 0.9187 - val_auc_1: 0.0000e+00 - val_loss: 0.2256
Epoch 90/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9208 - auc_1: 0.9740 - loss: 0.2049 - val_accuracy: 0.9219 - val_auc_1: 0.0000e+00 - val_loss: 0.219
Epoch 91/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9225 - auc_1: 0.9737 - loss: 0.2063 - val_accuracy: 0.9099 - val_auc_1: 0.0000e+00 - val_loss: 0.218
Epoch 92/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9234 - auc_1: 0.9735 - loss: 0.2083 - val_accuracy: 0.9146 - val_auc_1: 0.0000e+00 - val_loss: 0.2194
Epoch 93/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9203 - auc_1: 0.9742 - loss: 0.2052 - val_accuracy: 0.9099 - val_auc_1: 0.0000e+00 - val_loss: 0.2217
Epoch 94/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9196 - auc_1: 0.9733 - loss: 0.2077 - val_accuracy: 0.9078 - val_auc_1: 0.0000e+00 - val_loss: 0.2329
Epoch 95/100
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9243 - auc_1: 0.9730 - loss: 0.2092 - val_accuracy: 0.9260 - val_auc_1: 0.0000e+00 - val_loss: 0.211
Epoch 96/100
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
- Loss: 0.3086
- AUC: 0.9513
- Accuracy: 0.8840
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.91      0.85      0.88      1065
           1       0.86      0.91      0.89      1065

    accuracy                           0.88      2130
   macro avg       0.89      0.88      0.88      2130
weighted avg       0.89      0.88      0.88      2130
```

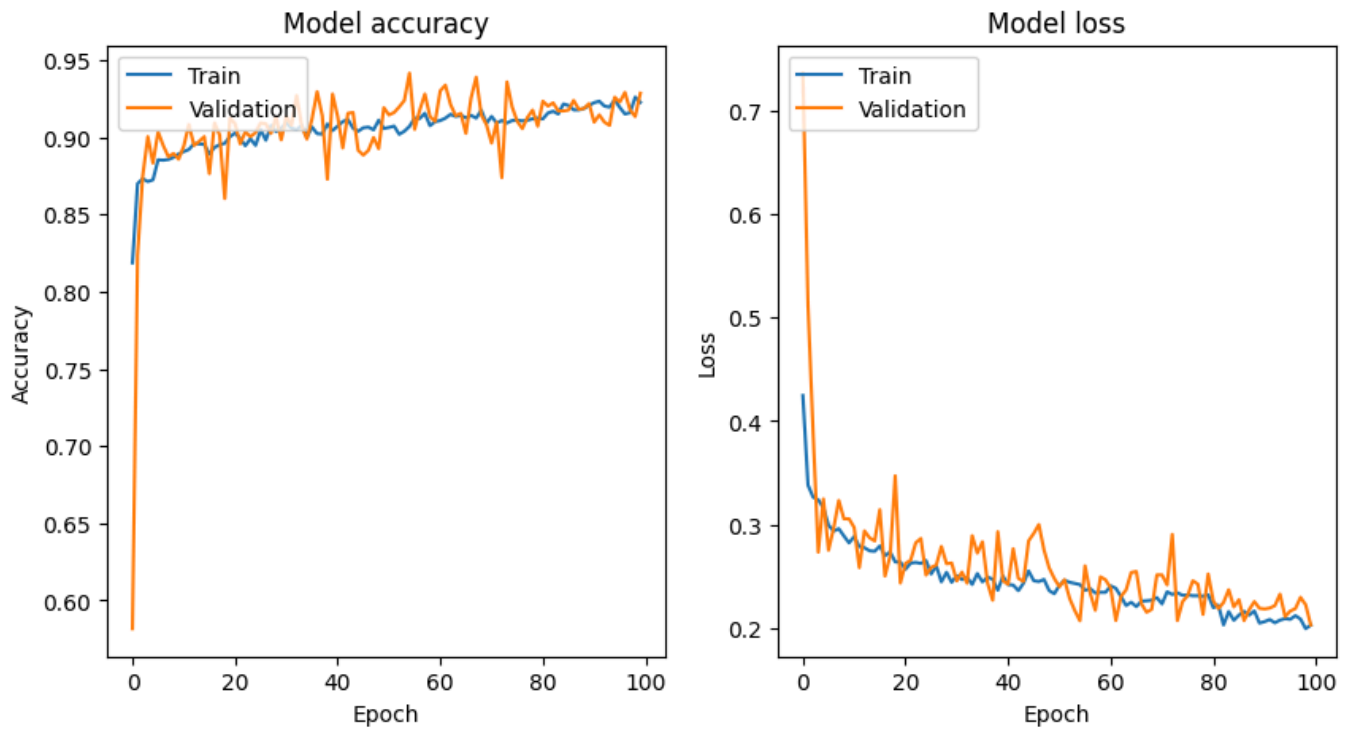
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
```

```
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

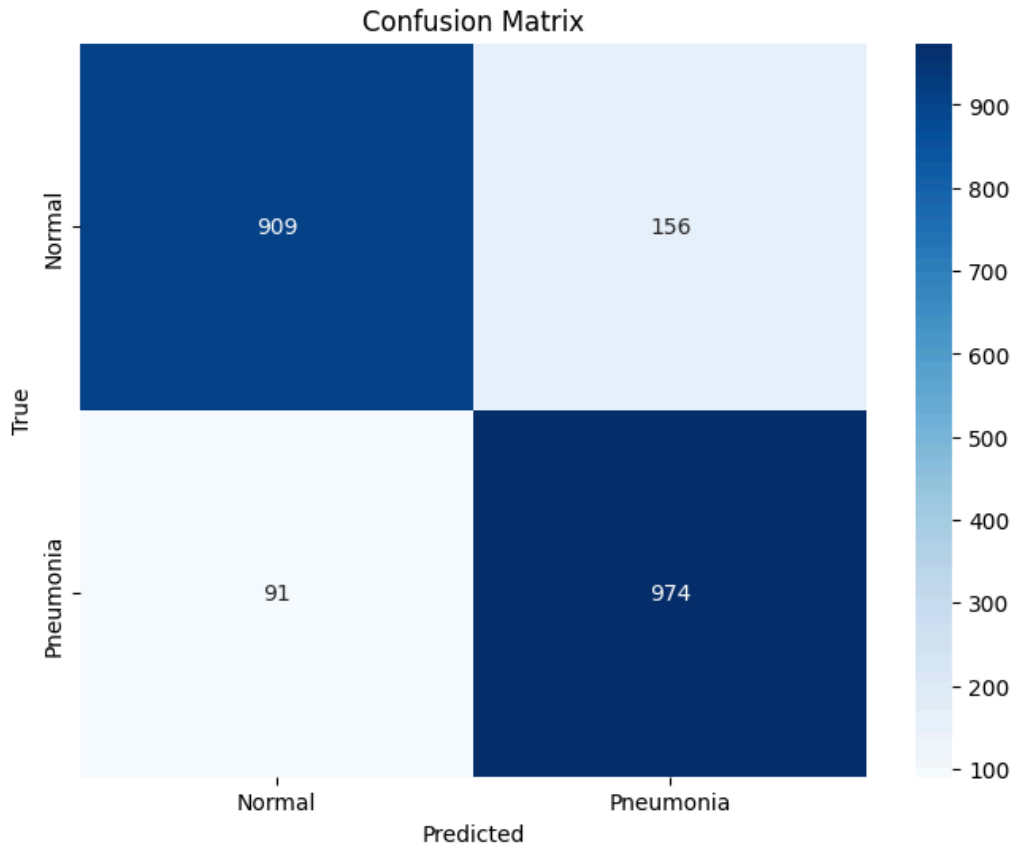
```
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
```

```
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```




```
model.save("Pnem_sim.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We re
```

```
from google.colab import files
files.download("Pnem_sim.h5")
```

```
import pickle

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
```

```
files.download("scaler.pkl")
```

CODE END HERE

Tes SVM

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```

```
--- SVM Model Evaluation ---
Accuracy: 0.8864

Classification Report:
              precision    recall  f1-score   support

           0       0.87      0.91      0.89      1065
           1       0.90      0.87      0.88      1065

    accuracy                           0.89      2130
```